Lecture 04 (B03): Generalization and Sequence Models

Bias-variance, double descent, RNN / LSTM / Transformer

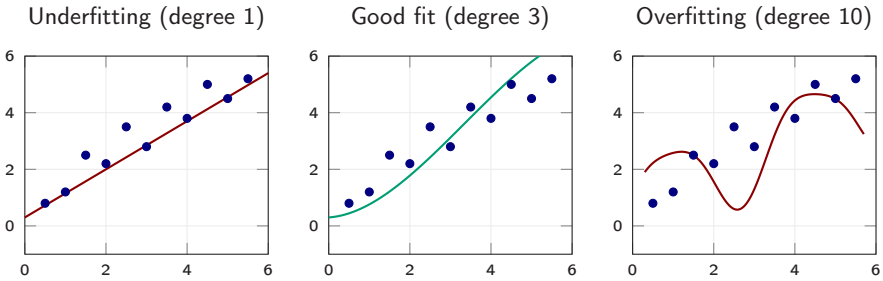
Simon Scheidegger

HEC, University of Lausanne

Part IV

Generalization and Regularization

Overfitting and Underfitting



- **Underfitting:** model too simple, high bias, cannot capture the pattern.
- **Overfitting:** model too complex, fits noise, poor generalization to new data.
- **Goal:** find the right complexity that generalizes well.

Train / Validation / Test Split

Training (∼70%)	Val (∼20%)	Test (∼10%)

Training set

Used to fit model parameters θ via optimization.

Validation set

Used for model selection: hyperparameters, architecture, early stopping.

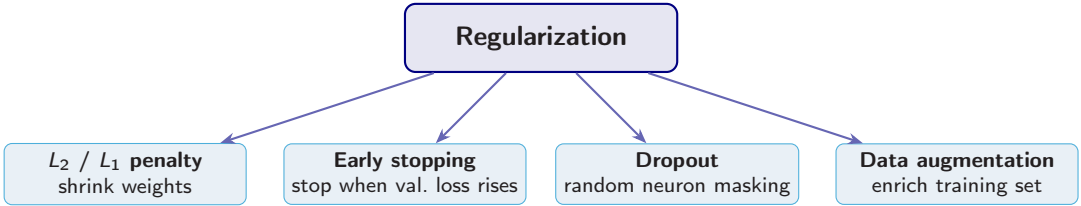
Test set

Touched **once** at the very end to report final performance.

Never use the test set for any decision during model development!

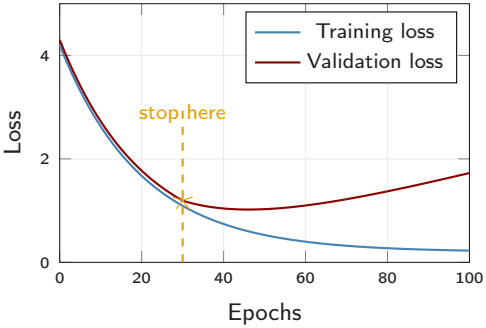
Regularization Techniques — Overview

Goal: bias the learner toward simpler hypotheses so it generalizes beyond the training sample.



The next slides cover **early stopping**, L_2, and **dropout** in detail; data augmentation is task-specific (images, time series) and is left as a reading pointer.

Early Stopping



Idea: monitor the validation loss during training. Stop when it starts increasing.

In practice:

1. After each epoch, evaluate loss on validation set.
2. Save model weights whenever validation loss improves.
3. If no improvement for p consecutive epochs ("patience"), stop training.
4. Return the saved best weights.

Early stopping is a simple and effective form of regularization.

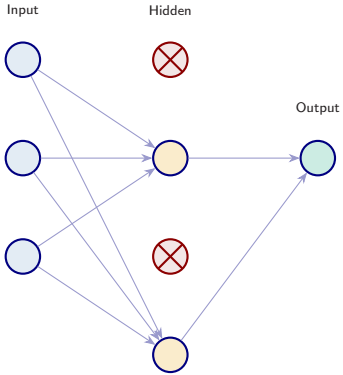
Regularization: L_2 Penalty and Dropout

L_2 Regularization (Weight Decay):
Add a penalty to the loss:

$$\tilde{J}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2$$

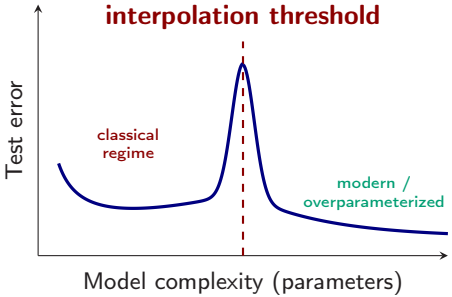
Encourages small weights $\Rightarrow$ simpler model.

Dropout (Srivastava et al., 2014): During training, randomly zero out each neuron with probability p (typically 0.5). Forces the network not to rely on any single neuron. The standard *inverted dropout* variant divides activations by $(1-p)$ during training, so no test-time rescaling is needed.



Dropout can be interpreted as training an **ensemble** of sub-networks that share weights.

Deep Double Descent



The classical U turns into a **second descent** once the model is overparameterized. Three regimes along the curve:

1. Classical ($n_{\text{par}} < n_{\text{data}}$): the familiar bias–variance U — small models underfit, larger ones chase noise.

2. Interpolation peak ($n_{\text{par}} \approx n_{\text{data}}$): model *just* fits the training set; among all zero-error solutions the optimizer's pick is unstable $\Rightarrow$ variance explodes.

3. Modern ($n_{\text{par}} \gg n_{\text{data}}$): gradient descent implicitly picks the minimum-norm / smoothest interpolator $\Rightarrow$ it generalizes *better*. More parameters help.

Belkin et al. (2019); Nakkiran et al. (2019).

Hands-on: `lecture_04_B03_03_Double_Descent.ipynb` — reproduce the curve with random Fourier features.

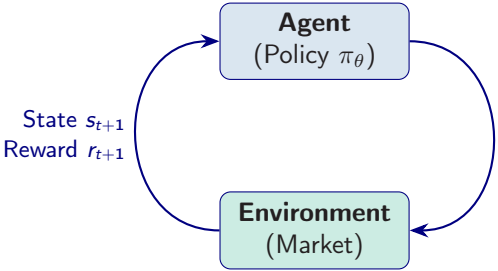
Part V

Advanced Architectures & Frontiers

Reinforcement Learning in Finance

The RL Loop:

- **Agent:** Portfolio manager / Algo trader
- **Environment:** The market (prices, order book)
- **State** s_t**:** Current prices, portfolio weights
- **Action** a_t**:** Buy/sell, rebalance
- **Reward** r_t**:** P&L, Sharpe ratio, utility



Why not in this course?

- RL finds *strategies* in a *fixed* environment.
- We focus on *finding the equilibrium* (the environment itself).
- BUT: Critical for hedging & execution.

Why Sequences? Memory Matters

Many real problems are intrinsically sequential:

- Next-word prediction: "Today we are having a class on deep _____"
- Time-series forecasting: inflation, volatility, returns conditional on history
- Audio / music: each sample predicted from thousands before (MuseNet, Wav2Vec)
- Econ application: Edgeworth cycles in gasoline prices (asymmetric sawtooth)

Why an MLP is not enough.

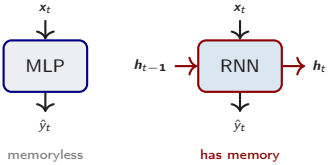
- MLP output is *memoryless*: $\hat{y}_t = f(\boldsymbol{x}_t)$.
- Same input $\Rightarrow$ same output, regardless of history.
- Sequences have *variable length*; MLPs have a fixed input size.
- We need a network that carries state across time.

Design goals (Rumelhart et al. 1986): **variable length** · **order** · **long-range dependence** · **parameter sharing across time**.

From MLP to Recurrence

Idea. Give the cell a persistent hidden state $\boldsymbol{h}_t \in \mathbb{R}^d$, fed back at every step.

- At step t, the cell sees *both* the new input $\boldsymbol{x}_t$ and yesterday's state $\boldsymbol{h}_{t-1}$.
- Same parameters $\{\boldsymbol{W}_{hh}, \boldsymbol{W}_{xh}, \boldsymbol{b}\}$ at every step: parameter sharing across time.
- The hidden state $\boldsymbol{h}_t$ is a learned summary of the past, the network's "memory".
- *One* cell processes *any* sequence length.



RNN is a dynamical system. Not a static function.

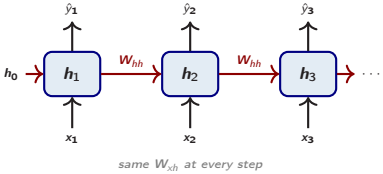
The RNN Cell, Unrolled Across Time

Two equations define the cell:

$$\boxed{\boldsymbol{h}_t = \tanh\big(\boldsymbol{W}_{hh}\,\boldsymbol{h}_{t-1} + \boldsymbol{W}_{xh}\,\boldsymbol{x}_t + \boldsymbol{b}_h\big)}$$

$$\hat{y}_t = \boldsymbol{W}_{hy}\,\boldsymbol{h}_t + \boldsymbol{b}_y$$

- $\boldsymbol{h}_0$ initialized to 0 (or learned).
- Weight sharing: $\boldsymbol{W}_{hh}, \boldsymbol{W}_{xh}, \boldsymbol{W}_{hy}$ do not depend on t.
- A *folded* loop with T steps $\equiv$ an *unfolded* feedforward graph of depth T with *shared* parameters.



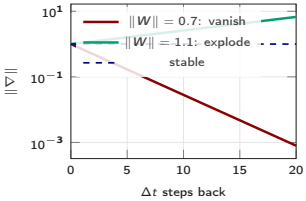
"Same function, many steps." Elman (1990); Rumelhart, Hinton & Williams (1986).

Training: Backpropagation Through Time (BPTT)

Idea. The unrolled RNN is a feedforward net of depth T with *shared* weights. Run backprop, *sum* weight-gradients across time.

Gradient through time. For the gradient of $\mathcal{L}_T$ w.r.t. an early state $\boldsymbol{h}_k$,

$$\frac{\partial \mathcal{L}_T}{\partial \boldsymbol{h}_k} = \frac{\partial \mathcal{L}_T}{\partial \boldsymbol{h}_T} \cdot \prod_{t=k+1}^{T} \underbrace{\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{h}_{t-1}}}_{= \boldsymbol{W}_{hh}^{\top} \operatorname{diag}(\tanh')} .$$



A long product of the same Jacobian:

- If $\|\boldsymbol{W}_{hh}\| > 1$: gradient explodes $\rightarrow$ NaN.
- If $\|\boldsymbol{W}_{hh}\| < 1$: gradient vanishes $\rightarrow$ no learning of long-range structure.

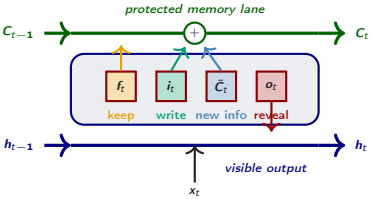
Remedies: gradient clipping, orthogonal init, truncated BPTT, and, most importantly, gated cells (LSTM, GRU).

LSTM: A Memory Conveyor Belt

Plain-language picture. A vanilla RNN rewrites its memory at every step. An LSTM keeps a dedicated cell state $\boldsymbol{C}_t$ that mostly flows forward unchanged.

At each date, the cell makes three soft decisions:

- Forget gate f_t: how much of the old memory survives?
- Input gate i_t: how much new information should be written?
- Output gate o_t: how much of memory should be exposed as $\boldsymbol{h}_t$?



$$\boldsymbol{C}_t = \boldsymbol{f}_t \odot \boldsymbol{C}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{C}}_t, \qquad \boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{C}_t)$$

- Key win: memory changes *additively*, so useful information can survive many steps.
- Useful for persistent asymmetric patterns, e.g. Edgeworth cycles.

Limits of Recurrence $\Rightarrow$ Attention

Even with LSTMs, two hard limits remain.

1. Sequential $\Rightarrow$ no parallelism.

- Step t must wait for step $t-1$.
- GPUs love parallelism $\Rightarrow$ RNN training is memory-bound and slow.
- Training modern LLMs on $\sim$1T tokens with RNNs would take *years*.

2. Path length blurs long-range signals.

- Information from position 1 reaches position T through $T-1$ hops.
- Each hop applies a matrix + nonlinearity $\Rightarrow$ fading memory.
- Gating helps but does not eliminate path-length decay.

RNN: $\mathcal{O}(T)$ path $\rightarrow$ LSTM: gated, still $\mathcal{O}(T)$ $\rightarrow$ **Attention: $\mathcal{O}(1)$ path**

Key move. Drop recurrence entirely: let every position read every other *directly*, in *parallel*. This is the Transformer (Vaswani et al. 2017).

Self-Attention: Search Inside the Sequence

Mental model. Updating token i is like running a search over the whole sequence.

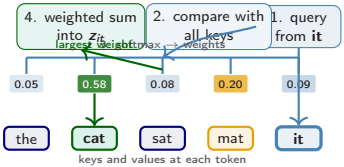
Each token creates three vectors:

- **Query** q_i: what am I looking for?
- **Key** k_j: what does token j advertise?
- **Value** v_j: what information can token j contribute?

Four-step recipe:

1. Add position information.
2. Compare query q_i with every key k_j.
3. Softmax the scores to get weights α_{ij}.
4. Return a weighted average of the values.

Worked attention pattern



Blue arrows: build the query and compare it to all keys.

Green arrows: use the weights to pull information, mainly from "cat", into z_{it}.

Displayed weights:

$$0.05 + 0.58 + 0.08 + 0.20 + 0.09 = 1.00.$$

From Self-Attention to a Transformer Block

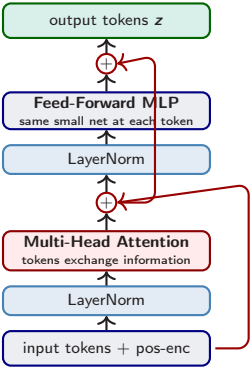
Matrix form:

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Why the extra ingredients?

- **Multi-head attention:** different heads can focus on different relations in parallel.
- **Positional encoding:** attention alone cannot see order, so we add a ruler.
- **Residuals + LayerNorm + MLP:** make deep stacks trainable and expressive.

Econ gloss: learnable kernel smoothing with learned features and learned similarity.



One block first mixes information *across tokens*, then transforms each token with an MLP, while skip connections keep deep stacks trainable.

RNN, LSTM, Transformer: What to Remember

RNN

- Hidden state = running summary of the past
- Best first mental model for sequence learning
- Hard to train on long histories

LSTM / GRU

- Add gates and a protected memory lane
- Strong baseline for specialized time-series tasks
- Still sequential, so no large-scale parallelism

Transformer

- Every token can read every other token directly
- Full parallelism across positions
- Default choice for long-context sequence modeling

Rule of Thumb

If the pattern is specialized and the history is moderate, an LSTM is still a strong baseline. If context is long and parallelism matters, start with a Transformer.

Sequence-model notebooks: `08_MLP_LSTM_Transformer_Edgeworth_Cycles` (the three-way comparison on the same task) and `09_Transformer_InContext_AR1` (advanced / optional).

Questions?

Simon Scheidegger

HEC, University of Lausanne

https://sischei.github.io

Materials: https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models